

HELPDESKAI

Scalable Enterprise Agentic AI Assistant for IT Support Automation

CAPSTONE DELIVERABLES

Scalable Enterprise Architectural Deployments of Agentic AI Solutions

2023103570 – Dhinesh Babu C M • Assignment Submission Document

02 October 2026

Executive Summary

HelpDeskAI is an agentic IT support assistant designed to automate common support operations through a controlled and auditable execution workflow. Unlike a conventional chatbot, the system interprets a user's intent, determines an appropriate action, selects a predefined capability from a Tool Registry, validates authorization, executes the operation, and records the execution lifecycle.

The solution adopts a layered service-oriented architecture with a React-based presentation layer, a Spring Boot application and agent orchestration layer, a controlled Tool Registry, and a relational persistence layer using SQLite for the current implementation. Security and governance are integrated into the execution path through role-based access control, programmatic guardrails, human-in-the-loop approval, and audit logging.

Scope Principle

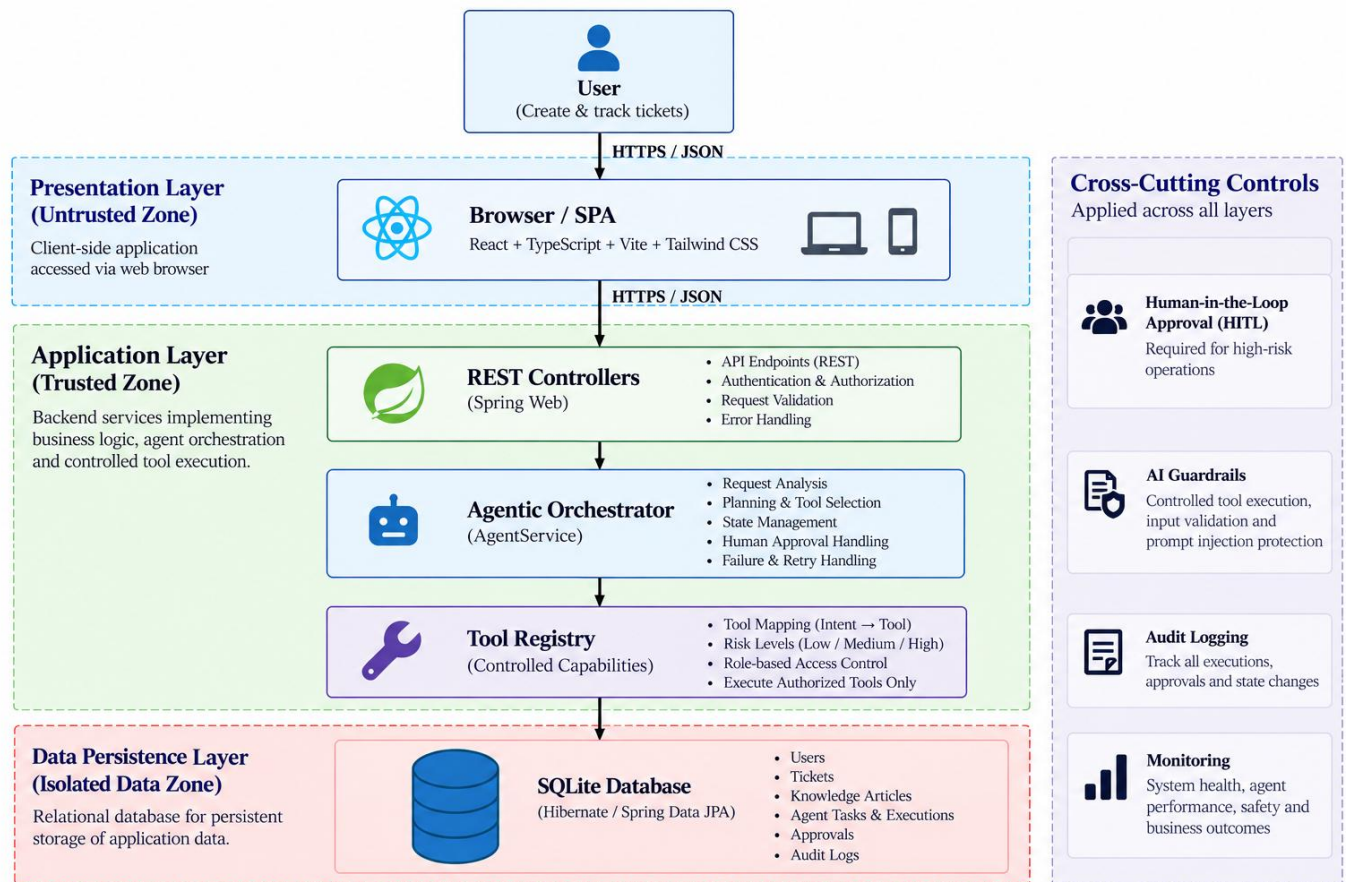
The implementation intentionally uses a single agent and a constrained set of tools. The objective is to demonstrate the architectural properties of an enterprise Agentic AI solution clearly and verifiably without introducing unnecessary distributed infrastructure.

1. Architecture Diagram

Layers, components, trust boundaries and integrations

The HelpDeskAI system conforms to an N-Tier Service-Oriented Architecture (SOA) augmented with a central Agentic Orchestration engine. The architecture emphasizes high cohesion, loose coupling, separation of concerns, explicit trust boundaries, controlled capability exposure, and traceable execution.

1.1 System Architecture



1.2 Presentation Layer

The presentation layer is implemented as a responsive client-side Single Page Application using React, TypeScript, Vite, and Tailwind CSS. It provides the user-facing AI Assistant, ticket management interfaces, knowledge access, approval workflows, execution history, and monitoring views. Communication with the backend is performed asynchronously through HTTP/REST requests.

1.3 Application and Orchestration Layer

The application layer is implemented as a Spring Boot application using Java 21. Spring Web exposes REST endpoints while the central AgentService coordinates the agent lifecycle. The agent follows an explicit finite-state execution model and uses a programmatic ToolRegistry to map supported intents to predefined operations.

- REST Controllers – expose application and agent APIs.
- AgentService – manages intent processing, planning, state transitions and execution.
- ToolRegistry – provides the controlled capability boundary for agent actions.
- Authorization services – validate role and operation permissions.
- Approval workflow – pauses high-risk operations until authorized human approval.
- Audit and monitoring services – capture execution and operational evidence.

1.4 Data and Persistence Layer

The current implementation uses SQLite as the relational persistence layer. Hibernate and Spring Data JPA provide object-relational mapping and repository access. The persistence model supports user management, support tickets, knowledge content, agent tasks, execution traces, approvals, tool executions, and audit records.

1.5 Trust Boundaries and Integrations

- Presentation trust boundary: the browser is treated as an untrusted client and cannot be relied upon to enforce authorization.
- Application trust boundary: the Spring Boot backend acts as the trusted enforcement and orchestration layer.
- Capability boundary: the agent cannot execute arbitrary code and can only invoke capabilities exposed by the Tool Registry.
- Human authorization boundary: high-risk operations are isolated from autonomous execution and require explicit approval.
- Persistence boundary: application data is accessed through backend services and repositories rather than direct client access.
- Optional AI integration boundary: an external AI provider may be connected through a server-side abstraction when required; the basic application remains functional without depending on an external provider.

2. Agent Workflow Design

Roles, states, tools, handoffs, approvals and failure paths

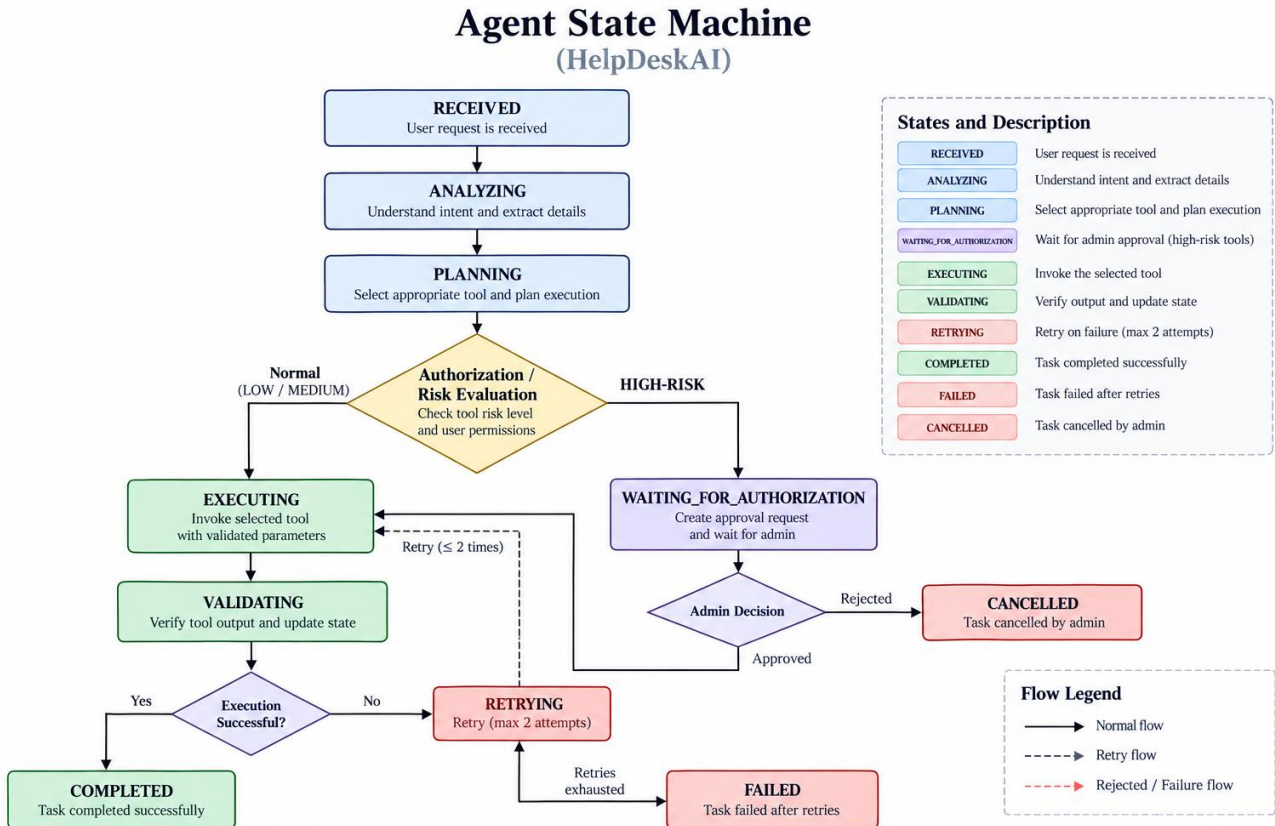
The HelpDeskAI agent operates using a deterministic state machine designed for safety, traceability, and controlled execution. Each task moves through explicit states and significant transitions are recorded for operational inspection.

2.1 Role-Based Access

The system implements role-based access control (RBAC) to determine the operations that can be performed by the authenticated user. Access privileges are organized according to the level of responsibility required for each operation.

- Standard Access: Allows users to submit IT support requests, interact with the AI assistant, and access permitted support functions.
- Support Access: Provides additional privileges for authorized support activities, including ticket management, updates, and resolution operations.
- Administrative Access: Provides elevated privileges for authorization of high-risk agent actions and access to administrative functions. Operational monitoring of the deployed application, including hosting, service availability, deployment status, and runtime-level observability, is handled through the Vercel and Render deployment platforms rather than being implemented as a separate infrastructure-management module within HelpDeskAI. The application itself provides the relevant agent execution, approval, and audit information required for application-level oversight.

2.2 Agent State Machine



2.3 Tool Registry

The agent is deliberately bounded by a strict Tool Registry rather than an open-ended execution mechanism. Each tool represents a predefined application capability and is associated with an appropriate risk level and authorization requirement.

- `search_knowledge` – retrieves approved knowledge-base information.
- `create_ticket` – creates a support ticket.
- `get_my_tickets` – retrieves tickets accessible to the current user.
- `get_ticket_status` – retrieves the status of a specified ticket.
- `update_ticket` – updates permitted ticket information.
- `resolve_ticket` – resolves an authorized support ticket.
- `close_ticket` – performs a high-risk ticket closure operation requiring approval.

2.4 Human-in-the-Loop Handoff

When the agent identifies a HIGH-risk operation, autonomous execution is suspended. The system transitions to `WAITING_FOR_AUTHORIZATION`, creates an Approval record, and presents the pending operation to an authorized administrator through the Approvals interface.

1. Agent determines that a requested operation requires elevated authorization.
2. Backend verifies that the requester is permitted to initiate the operation.
3. An approval request is generated and persisted.
4. Agent execution pauses in `WAITING_FOR_AUTHORIZATION`.
5. An ADMIN reviews the operation and approves or rejects it.

6. Approved operations proceed through the Tool Registry.
7. Rejected operations terminate safely and transition to CANCELLED.

2.5 Failure Paths

- Unsupported intent: the agent does not invoke an unregistered capability and returns a controlled fallback response.
- Authorization failure: the requested operation is denied and recorded in the audit trail.
- Tool execution failure: the operation is recorded and may enter the bounded retry path.
- Repeated execution failure: the task transitions to FAILED and the failure is surfaced to the appropriate user/administrator.
- Approval rejection: the task transitions to CANCELLED without executing the high-risk operation.

3. Deployment Strategy

Runtime, environments, hosting, resilience and release

The current deployment separates the presentation and application tiers. The React frontend is deployed on Vercel, while the Spring Boot backend is deployed on Render. This deployment model preserves the architectural separation between the client application and the trusted backend orchestration layer.

Cold Start: Since the backend is hosted on Render's free tier, the first request after inactivity may experience a delay while the service starts up. Subsequent requests operate normally.

3.1 Frontend Deployment – Vercel

The React/Vite frontend is hosted on Vercel. Vercel serves the compiled single-page application and provides the public entry point for users. The frontend communicates with the Spring Boot backend through HTTPS REST requests.

Frontend Deployment URL: [HelpDeskAI Website Link](#)

3.2 Backend Deployment – Render

The Spring Boot backend is deployed on Render. The backend exposes the REST APIs and contains the agent orchestration, Tool Registry, authorization, approval workflow, business logic, audit handling, and persistence integration.

Backend Deployment URL: [Render Backend URL](#)

3.3 Application Access

Live Application URL: [HelpDeskAI Website Link](#)

3.4 Deployment Model

- Presentation runtime: React/Vite application hosted on Vercel.
- Application runtime: Java 21 Spring Boot service hosted on Render.
- Communication: HTTPS REST/JSON between the frontend and backend.
- Persistence: SQLite for the current implementation.
- Environment configuration: deployment-specific environment variables are used for configurable values and secrets.

- Release approach: frontend and backend can be deployed independently while preserving the API contract.

3.5 Enterprise Deployment Evolution

For a larger production environment, the current deployment can evolve without changing the core governance model. The frontend can remain independently hosted, while the backend can be containerized and horizontally scaled behind an API gateway or ingress.

- Move from SQLite to PostgreSQL or another production relational database.
- Containerize the Spring Boot backend and frontend where operationally appropriate.
- Introduce horizontal scaling for stateless backend instances.
- Use centralized logging, metrics and distributed tracing.
- Use managed secret storage for external AI and infrastructure credentials.
- Introduce health checks, controlled rollouts and automated rollback mechanisms.
- Adopt Kubernetes only when the application's scale and operational requirements justify it.

4. Security Model

Identity, authorization, secrets, privacy, guardrails and audit

Security is enforced as a backend responsibility and is integrated directly into the agent execution path. The system follows the principle of least privilege by restricting agent capabilities to predefined tools and requiring elevated approval for high-risk actions.

4.1 Identity and Authorization

- API requests are authenticated before protected operations are processed.
- Tools are explicitly associated with the roles permitted to invoke them.
- The frontend does not act as the final authorization boundary; the backend validates permissions.
- High-risk tools such as `close_ticket` require elevated authorization and human approval.

4.2 Programmatic Guardrails

The agent cannot execute arbitrary code or arbitrary database operations. It can only invoke capabilities explicitly exposed by the Tool Registry. The intent-to-tool mapping is constrained by application logic, and inputs are validated before sensitive operations are executed.

- No arbitrary code execution through the agent.
- No unrestricted database access from the agent.
- Only registered tools can be invoked.
- Role and risk checks are performed before execution.
- High-risk operations require human authorization.
- Unsupported requests result in a controlled fallback rather than an invented capability.

4.3 Prompt Injection and Input Safety

The system reduces the impact of prompt-injection-style requests by separating natural language intent handling from executable capabilities. User input cannot create new tools or dynamically grant permissions. The backend determines whether an intended operation maps to a registered capability and whether the requesting role is authorized to perform it.

4.4 Auditability

Agent state transitions and tool executions are recorded in the agent execution and audit records. The audit trail provides a traceable chain of activity showing the initiating user, relevant operation, execution status, and approval information where applicable. This supports accountability and post-execution review.

Audit Accuracy

The audit trail should be described as traceable and tamper-resistant only to the extent supported by the implemented storage and controls. A cryptographically signed or hashed audit chain should be added before claiming cryptographic verification.

4.5 Privacy and Secrets

- Database credentials, API keys and other sensitive configuration are supplied through environment variables rather than hardcoded in source code.
- User requests are processed through the backend application boundary.
- The frontend should not contain server-side secrets or provider credentials.
- Access to user and ticket information is restricted according to role and application permissions.

5. Monitoring Dashboard Design

Health, trace, quality, safety, cost and business outcomes

The deployed HelpDeskAI application uses the monitoring and observability capabilities provided by its deployment platforms, **Vercel for the frontend and Render for the backend**. This approach provides visibility into application availability, deployment status, runtime health, errors, and resource usage without introducing a separate monitoring infrastructure within the HelpDeskAI application.

Application-level information such as agent execution states, approvals, tool activity, and audit records is maintained by the HelpDeskAI backend and can be used to understand agent behavior and support operations.

5.1 Deployment and System Health

The deployment platforms provide operational visibility into the availability and runtime behavior of the deployed components.

- **Vercel:** Frontend deployment status, availability, build/deployment information, and frontend runtime activity.
- **Render:** Backend service status, application availability, deployment information, logs, and runtime resource usage.
- Backend API errors and application logs can be inspected through the deployed backend service.
- Frontend and backend deployment status can be monitored independently.

5.2 Agent Execution Traceability

Agent-level traceability is provided through the application's **agent execution and audit records**, rather than through a separate external tracing system.

The execution lifecycle can be inspected using states such as:

RECEIVED → ANALYZING → PLANNING → EXECUTING → VALIDATING → COMPLETED

with additional states for:

WAITING_FOR_AUTHORIZATION, RETRYING, FAILED, and CANCELLED.

The application records relevant execution information such as:

- Agent execution status.
- Selected tool and tool execution status.
- Approval requirements and decisions.
- Execution failures and retry activity.
- Audit events associated with important operations.

5.3 Safety and Compliance Monitoring

Safety-related monitoring is primarily supported through the application's **RBAC, Tool Registry, approval workflow, and audit records**.

- High-risk operations can be identified through approval requests.
- Authorization failures can be reviewed through application records and backend logs.
- Approved and rejected high-risk operations are recorded.
- Tool execution activity provides visibility into actions performed by the agent.
- Audit records provide traceability for sensitive operations.

5.4 Quality and Business Outcomes

Application-level support information can be used to evaluate the effectiveness of the agentic workflow.

Relevant indicators include:

- Successful and failed agent executions.
- Ticket creation and resolution activity.
- Validation failures.
- Time-to-resolution (TTR), where sufficient execution data is available.
- Human approval turnaround time.
- AI-assisted resolution or deflection, where sufficient application data exists.

These are treated as **application-level operational indicators**, rather than claiming the existence of a separate built-in analytics platform.

5.5 Cost and Resource Monitoring

Infrastructure-level resource and service information is monitored through the respective deployment platforms.

- **Vercel:** Frontend deployment and hosting usage information.
- **Render:** Backend service runtime and resource usage information.
- If an external AI provider is integrated, its API usage and token consumption can be monitored through the provider's respective platform.
- When the application operates in deterministic/demo mode, no external AI inference cost is incurred.

6. Conclusion

HelpDeskAI demonstrates a compact but complete Agentic AI architecture for IT support. The system combines a modern web presentation layer, trusted Spring Boot backend orchestration, a controlled Tool Registry, explicit state management, human-in-the-loop approval, auditability, and operational monitoring.

The architecture is intentionally designed so that autonomy is constrained by explicit capabilities and governance. This makes the system suitable for demonstrating the five required capstone artifacts while maintaining a clear path toward enterprise-scale deployment and operational maturity.

7. Deployment Link

Live Application: [HelpDeskAI Website Link](#)

Cold Start: Since the backend is hosted on Render's free tier, the first request after inactivity may experience a delay while the service starts up. Subsequent requests operate normally.